

Uniwersytet Bielsko-Bialski

Wydział Budowy Maszyn i Informatyki

Nauka o danych I

Ćwiczenie numer:

1

Temat ćwiczenia:

Ustalenia platformu Jupyter Uzycie biblioteki pandas

Radosław Walczak

Nr albumu: 60038

1. Wstęp

Celem ćwiczenia było nabycie podstawowej znajomości języka Python oraz biblioteki pandas poprzez wczytanie zbioru danych, utworzenie ramek danych i wykonanie na nich podstawowych operacji: przeglądania, filtrowania, sortowania, grupowania, tworzenia tabel przestawnych i wykresów.

Ćwiczenie wykonano w wariancie 7: Global Burden of Disease Study 2019 (GBD 2019) – Smoking Tobacco Use Prevalence 1990-2019. Dane pobrano z repozytorium IHME (Global Health Data Exchange): https://ghdx.healthdata.org/record/ihme-data/gbd-2019-smoking-tobacco-use-prevalence-1990-2019. Wykorzystano plik „Age-standardized smoking prevalence estimates” `IHME_GBD_2019_SMOKING_TOB_1990_2019_PREV_Y2021M05D27.CSV`.

Plik zawiera 20 970 wierszy. Każdy wiersz to oszacowanie standaryzowanej wiekowo częstości palenia tytoniu dla jednej lokalizacji (231 krajów i regionów), jednej płci i jednego roku z lat 1990-2019. Do analizy wybrano 7 kolumn:

Kolumna	**Opis**
location	kraj lub region (np. Poland, Global)
sex	płeć: Male, Female, Both (obie płcie)
age_group	grupa wiekowa – zawsze 15+ years
year	rok 1990-2019
val	częstość palenia jako ułamek populacji (0-1)
lower / upper	dolna i górna granica przedziału niepewności 95%

Zadanie wykonano w notatniku Jupyter, w języku Python 3 z bibliotekami pandas i matplotlib. Notatnik `lab1_wariant7.ipynb` umieszczono w repozytorium:

https://github.com/RadoslawWalczak63/nod_lab1

2. Przebieg ćwiczenia

2.1. Przygotowanie i wczytanie danych

Krok 1. Ładowanie biblioteki pandas

Zaimportowano bibliotekę pandas pod skróconą nazwą pd.

```
import pandas as pd
```

Krok 2. Tworzenie ramki danych ze słownika

Utworzono ramkę ze słownika, w którym klucze to nazwy kolumn, a listy to ich wartości.

```
dane_slownik = {"kraj": ["Poland", "Germany", "France", "Spain"],
                "rok": [2019, 2019, 2019, 2019],
                "czestosc_palenia": [0.245, 0.221, 0.287, 0.262]}
df_slownik = pd.DataFrame(dane_slownik)
```

Wynik w konsoli:

```
kraj   rok  czestosc_palenia
0   Poland  2019            0.245
1  Germany  2019            0.221
2   France  2019            0.287
3    Spain  2019            0.262
```

Wynik: ramka 4 × 3.

Krok 3. Wczytanie ramki danych z pliku CSV

Plik wczytano funkcją `read_csv()`, skrócono nazwy kolumn metodą `rename()` i wybrano potrzebne kolumny.

```
df = pd.read_csv(plik)
df = df.rename(columns={"location_name": "location", "sex_name": "sex",
                        "age_group_name": "age_group", "year_id": "year"})
df = df[["location", "sex", "age_group", "year", "val", "upper", "lower"]]
```

Wynik w konsoli:

```
location     sex  age_group  year       val     upper     lower
0   Global    Male  15+ years  1990  0.450607  0.453930  0.446915
1   Global  Female  15+ years  1990  0.106411  0.108623  0.104415
2   Global    Both  15+ years  1990  0.278179  0.280235  0.276236
3   Global    Male  15+ years  1991  0.447212  0.450214  0.443670
4   Global  Female  15+ years  1991  0.105164  0.107276  0.103293
```

Krok 4. Tworzenie ramki danych z listy list

Każda wewnętrzna lista to jeden wiersz, nazwy kolumn podano w parametrze `columns`.

```
lista_list = [["Poland", "Male", 2019, 0.31], ["Poland", "Female", 2019, 0.19], ...]
df_lista = pd.DataFrame(lista_list, columns=["kraj", "plec", "rok", "czestosc"])
```

Wynik w konsoli:

```
kraj    plec   rok  czestosc
0   Poland    Male  2019      0.31
1   Poland  Female  2019      0.19
2  Germany    Male  2019      0.27
3  Germany  Female  2019      0.18
```

Krok 5. Transponowanie

Atrybut `.T` zamienia wiersze z kolumnami – ramka 4 × 4 po transpozycji ma w wierszach nazwy dawnych kolumn.

```
df_lista.T
```

Wynik w konsoli:

```
0       1        2        3
kraj      Poland  Poland  Germany  Germany
plec        Male  Female     Male   Female
rok         2019    2019     2019     2019
czestosc    0.31    0.19     0.27     0.18
```

2.2. Przegląd danych

Kroki 6-7. Pierwsze i ostatnie 10 wierszy

Metody `head(10)` i `tail(10)` wyświetlają początek i koniec ramki. Pierwsze wiersze dotyczą lokalizacji Global w roku 1990.

```
df.head(10)
df.tail(10)
```

Wynik w konsoli:

```
location     sex  age_group  year       val     upper     lower
0   Global    Male  15+ years  1990  0.450607  0.453930  0.446915
1   Global  Female  15+ years  1990  0.106411  0.108623  0.104415
2   Global    Both  15+ years  1990  0.278179  0.280235  0.276236
3   Global    Male  15+ years  1991  0.447212  0.450214  0.443670
4   Global  Female  15+ years  1991  0.105164  0.107276  0.103293
5   Global    Both  15+ years  1991  0.275964  0.277921  0.274053
6   Global    Male  15+ years  1992  0.443359  0.446333  0.440023
7   Global  Female  15+ years  1992  0.103951  0.105943  0.102128
8   Global    Both  15+ years  1992  0.273516  0.275395  0.271688
9   Global    Male  15+ years  1993  0.439086  0.442012  0.435972


      location     sex  age_group  year       val     upper     lower
20960    Sudan    Both  15+ years  2016  0.107539  0.115340  0.099871
20961    Sudan    Male  15+ years  2017  0.195406  0.211129  0.180533
20962    Sudan  Female  15+ years  2017  0.019547  0.025471  0.014919
20963    Sudan    Both  15+ years  2017  0.107313  0.115101  0.099585
20964    Sudan    Male  15+ years  2018  0.195293  0.210935  0.180295
20965    Sudan  Female  15+ years  2018  0.019498  0.025394  0.014783
20966    Sudan    Both  15+ years  2018  0.107243  0.115060  0.099628
20967    Sudan    Male  15+ years  2019  0.195406  0.211303  0.179990
20968    Sudan  Female  15+ years  2019  0.019456  0.025376  0.014643
20969    Sudan    Both  15+ years  2019  0.107282  0.115526  0.099393
```

Krok 8. Informacje o ramce danych

Metoda `info()` pokazała 20 970 wierszy, 7 kolumn: 3 tekstowe (location, sex, age_group), 1 całkowitą (year) i 3 zmiennoprzecinkowe (val, upper, lower), bez wartości pustych.

```
df.info()
```

Wynik w konsoli:

```
<class 'pandas.DataFrame'>
RangeIndex: 20970 entries, 0 to 20969
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   location   20970 non-null  str
 1   sex        20970 non-null  str
 2   age_group  20970 non-null  str
 3   year       20970 non-null  int64
 4   val        20970 non-null  float64
 5   upper      20970 non-null  float64
 6   lower      20970 non-null  float64
dtypes: float64(3), int64(1), str(3)
memory usage: 1.1 MB
```

Krok 9. Liczba wierszy i kolumn

Atrybut shape zwraca krotkę (wiersze, kolumny).

```
df.shape
```

Wynik w konsoli:

```
(20970, 7)
```

Wynik: (20970, 7).

Krok 10. Statystyki kolumn liczbowych

Metoda `describe()` zwraca liczbę wartości, średnią, odchylenie standardowe, minimum, kwartyle i maksimum, a `nunique()` liczbę wartości niepowtarzalnych.

```
df.describe()
df.select_dtypes("number").nunique()
```

Wynik w konsoli:

```
year           val         upper         lower
count  20970.000000  20970.000000  20970.000000  20970.000000
mean    2004.500000      0.220575      0.233756      0.208045
std        8.655648      0.138533      0.141806      0.135304
min     1990.000000      0.006671      0.008809      0.005015
25%     1997.000000      0.104265      0.114119      0.095013
50%     2004.500000      0.214082      0.227842      0.199491
75%     2012.000000      0.316076      0.330792      0.301007
max     2019.000000      0.656047      0.668383      0.644400
year        30
val      20790
upper    20789
lower    20790
dtype: int64
```

Krok 11. Statystyki kolumn kategoryzowanych

Dla kolumn tekstowych `describe(exclude="number")` zwraca liczbę unikalnych wartości (unique), najczęstszą wartość (top) i jej liczność (freq).

```
df.describe(exclude="number")
```

Wynik w konsoli:

```
location    sex  age_group
count                        20970  20970      20970
unique                         231      3          1
top     North Africa and Middle East   Male  15+ years
freq                           180   6990      20970
```

Krok 12. Usunięcie brakujących wartości

Braki sprawdzono metodą `isna().sum()` i usunięto metodą `dropna()`.

```
df.isna().sum()
df = df.dropna().reset_index(drop=True)
```

Wynik w konsoli:

```
location     0
sex          0
age_group    0
year         0
val          0
upper        0
lower        0
dtype: int64
(20970, 7)
```

Wynik: zbiór nie zawiera brakujących wartości, rozmiar pozostał (20970, 7).

2.3. Wybór i filtrowanie danych

Krok 13. Wybór wierszy i kolumn po nazwach i indeksach

Pokazano wybór kolumn po nazwie (`df["kolumna"]`), po etykietach (`loc`), po pozycjach (`iloc`) oraz pojedynczej komórki (`at`, `iat`).

```
df["location"]
df[["location", "year", "val"]]
df.loc[0:4, ["location", "sex", "val"]]
df.iloc[0:5, 0:3]
df.at[0, "val"]     # 0.4506
df.iat[0, 0]        # Global
```

Wynik w konsoli:

```
0    Global
1    Global
2    Global
3    Global
4    Global
Name: location, dtype: str
  location  year       val
0   Global  1990  0.450607
1   Global  1990  0.106411
2   Global  1990  0.278179
3   Global  1991  0.447212
4   Global  1991  0.105164
  location     sex       val
0   Global    Male  0.450607
1   Global  Female  0.106411
2   Global    Both  0.278179
3   Global    Male  0.447212
4   Global  Female  0.105164
  location     sex  age_group
0   Global    Male  15+ years
1   Global  Female  15+ years
2   Global    Both  15+ years
3   Global    Male  15+ years
4   Global  Female  15+ years
0.450606958
Global
```

Krok 14. Wybór wierszy pod jednym warunkiem

Wybrano wiersze z roku 2019 za pomocą maski logicznej.

```
df[df["year"] == 2019]
```

Wynik w konsoli:

```
location     sex  age_group  year  \
87                                 Global    Male  15+ years  2019
88                                 Global  Female  15+ years  2019
89                                 Global    Both  15+ years  2019
177  Southeast Asia, East Asia, and Oceania    Male  15+ years  2019
178  Southeast Asia, East Asia, and Oceania  Female  15+ years  2019
179  Southeast Asia, East Asia, and Oceania    Both  15+ years  2019
267                            East Asia    Male  15+ years  2019
268                            East Asia  Female  15+ years  2019
269                            East Asia    Both  15+ years  2019
357                                China    Male  15+ years  2019
```

```
         val     upper     lower
87   0.326673  0.330424  0.322837
...
```

Krok 15. Wybór wierszy pod kilkoma warunkami

Warunki połączono operatorem &, a w drugim wariancie użyto metody `query()`.

```
df[(df["year"] == 2019) & (df["sex"] == "Male") & (df["val"] > 0.4)]
df.query("year == 2019 and sex == 'Female' and val > 0.2")
```

Wynik w konsoli:

```
location   sex  age_group  year       val  \
177   Southeast Asia, East Asia, and Oceania  Male  15+ years  2019  0.494283
267                               East Asia  Male  15+ years  2019  0.494572
357                                   China  Male  15+ years  2019  0.497344
447    Democratic People's Republic of Korea  Male  15+ years  2019  0.435507
627                           Southeast Asia  Male  15+ years  2019  0.482342
717                                 Cambodia  Male  15+ years  2019  0.406229
807                                Indonesia  Male  15+ years  2019  0.583269
897         Lao People's Democratic Republic  Male  15+ years  2019  0.491010
987                                 Malaysia  Male  15+ years  2019  0.402585
1077                                Maldives  Male  15+ years  2019  0.465260

         upper     lower
177   0.504230  0.484354
...
                            location     sex  age_group  year       val  \
1888                        Kiribati  Female  15+ years  2019  0.351282
2068  Micronesia (Federated States of)  Female  15+ years  2019  0.363633
3598                   Central Europe  Female  15+ years  2019  0.258603
3778            Bosnia and Herzegovina  Female  15+ years  2019  0.305410
3868                          Bulgaria  Female  15+ years  2019  0.324911
3958                           Croatia  Female  15+ years  2019  0.326480
4048                           Czechia  Female  15+ years  2019  0.232210
4138                           Hungary  Female  15+ years  2019  0.264520
4228                   North Macedonia  Female  15+ years  2019  0.310483
4318                        Montenegro  Female  15+ years  2019  0.326599

         upper     lower
1888  0.385660  0.315212
...
```

Wynik: 56 lokalizacji, w których w 2019 r. pali ponad 40% mężczyzn, oraz 45 lokalizacji, w których pali ponad 20% kobiet.

Kroki 16-17. Wiersze zawierające / nie zawierające słowa

Użyto metody `str.contains()`, a negację warunku zapisano operatorem ~.

```
df[df["location"].str.contains("Poland")]
df[~df["location"].str.contains("Republic")]
```

Wynik w konsoli:

```
location     sex  age_group  year       val     upper     lower
4320   Poland    Male  15+ years  1990  0.534484  0.549658  0.520634
4321   Poland  Female  15+ years  1990  0.297057  0.312776  0.279601
4322   Poland    Both  15+ years  1990  0.411484  0.422280  0.400479
4323   Poland    Male  15+ years  1991  0.523955  0.538589  0.510206
4324   Poland  Female  15+ years  1991  0.294519  0.310727  0.277411
4325   Poland    Both  15+ years  1991  0.405091  0.415698  0.394280
4326   Poland    Male  15+ years  1992  0.513121  0.527595  0.499073
4327   Poland  Female  15+ years  1992  0.291945  0.309717  0.274194
```

```
4328   Poland    Both  15+ years  1992  0.398517  0.409643  0.387606
4329   Poland    Male  15+ years  1993  0.501397  0.516145  0.487675


  location     sex  age_group  year       val     upper     lower
0   Global    Male  15+ years  1990  0.450607  0.453930  0.446915
1   Global  Female  15+ years  1990  0.106411  0.108623  0.104415
2   Global    Both  15+ years  1990  0.278179  0.280235  0.276236
3   Global    Male  15+ years  1991  0.447212  0.450214  0.443670
4   Global  Female  15+ years  1991  0.105164  0.107276  0.103293
5   Global    Both  15+ years  1991  0.275964  0.277921  0.274053
6   Global    Male  15+ years  1992  0.443359  0.446333  0.440023
7   Global  Female  15+ years  1992  0.103951  0.105943  0.102128
8   Global    Both  15+ years  1992  0.273516  0.275395  0.271688
9   Global    Male  15+ years  1993  0.439086  0.442012  0.435972
```

2.4. Modyfikacja kolumn i zapis do pliku

Krok 18. Utworzenie kolumn na podstawie istniejących

Dodano częstość w procentach oraz szerokość przedziału niepewności.

```
df["val_proc"] = df["val"] * 100
df["szerokosc"] = df["upper"] - df["lower"]
```

Wynik w konsoli:

```
location     sex  age_group  year       val     upper     lower   val_proc  \
0   Global    Male  15+ years  1990  0.450607  0.453930  0.446915  45.060696
1   Global  Female  15+ years  1990  0.106411  0.108623  0.104415  10.641115
2   Global    Both  15+ years  1990  0.278179  0.280235  0.276236  27.817878
3   Global    Male  15+ years  1991  0.447212  0.450214  0.443670  44.721154
4   Global  Female  15+ years  1991  0.105164  0.107276  0.103293  10.516446


   szerokosc
0   0.007015
1   0.004208
2   0.003998
3   0.006544
4   0.003983
```

Kroki 19-20. Usunięcie kolumny i zmiana nazwy

Metodą `drop()` usunięto kolumnę szerokosc, a metodą `rename()` zmieniono nazwę val_proc na prevalence_percent.

```
df = df.drop(columns=["szerokosc"])
df = df.rename(columns={"val_proc": "prevalence_percent"})
```

Wynik w konsoli:

```
Index(['location', 'sex', 'age_group', 'year', 'val', 'upper', 'lower',
       'val_proc'],
      dtype='str')


Index(['location', 'sex', 'age_group', 'year', 'val', 'upper', 'lower',
       'prevalence_percent'],
      dtype='str')
```

Krok 21. Zapis ramki danych do pliku CSV

Ramkę zapisano do pliku CSV na komputerze.

```
df.to_csv("wynik_smoking.csv", index=False)
```

2.5. Statystyki, wartości unikatowe i sortowanie

Krok 22. Średnia, maksimum i minimum kolumny

Użyto metod `mean()`, `max()` i `min()` dla kolumny val.

```
df["val"].mean()   # 0.2206
df["val"].max()    # 0.6560
df["val"].min()    # 0.0067
```

Wynik w konsoli:

```
Średnia: 0.22057514106728666
Maksimum: 0.656046804
Minimum: 0.006671118
```

Kroki 23-25. Liczba wierszy, wartości unikatowe, liczności

Funkcja `len()` zwróciła 20 970 wierszy, `unique()` dla kolumny sex zwróciła Male, Female, Both, a `value_counts()` pokazała po 6 990 rekordów dla każdej płci.

```
len(df)
df["sex"].unique()
df["sex"].value_counts()
```

Wynik w konsoli:

```
20970


<StringArray>
['Male', 'Female', 'Both']
Length: 3, dtype: str


sex
Male      6990
Female    6990
Both      6990
Name: count, dtype: int64
```

Krok 26. Sortowanie rosnąco i malejąco

Metoda `sort_values()` z parametrem `ascending`.

```
df.sort_values("val")
df.sort_values("val", ascending=False)
```

Wynik w konsoli:

```
location     sex  age_group  year       val     upper     lower  \
15811  Eritrea  Female  15+ years  2010  0.006671  0.008813  0.005015
15814  Eritrea  Female  15+ years  2011  0.006722  0.008809  0.005059
15817  Eritrea  Female  15+ years  2012  0.006776  0.008892  0.005089
15820  Eritrea  Female  15+ years  2013  0.006813  0.008929  0.005093
15823  Eritrea  Female  15+ years  2014  0.006849  0.008858  0.005142


       prevalence_percent
15811            0.667112
15814            0.672247
15817            0.677594
15820            0.681277
15823            0.684913
      location   sex  age_group  year       val     upper     lower  \
```

```
1845  Kiribati  Male  15+ years  2005  0.656047  0.667851  0.644400
1842  Kiribati  Male  15+ years  2004  0.656020  0.668383  0.643124
1848  Kiribati  Male  15+ years  2006  0.655416  0.666569  0.643675
1839  Kiribati  Male  15+ years  2003  0.654347  0.667074  0.640243
1851  Kiribati  Male  15+ years  2007  0.653724  0.665278  0.642580

      prevalence_percent
1845           65.604680
1842           65.601994
1848           65.541569
1839           65.434741
1851           65.372370
```

Krok 27. 10 największych i 10 najmniejszych wartości

Użyto metod `nlargest()` i `nsmallest()`.

```
df.nlargest(10, "val")
df.nsmallest(10, "val")
```

Wynik w konsoli:

```
location   sex  age_group  year       val     upper     lower  \
1845     Kiribati  Male  15+ years  2005  0.656047  0.667851  0.644400
1842     Kiribati  Male  15+ years  2004  0.656020  0.668383  0.643124
1848     Kiribati  Male  15+ years  2006  0.655416  0.666569  0.643675
1839     Kiribati  Male  15+ years  2003  0.654347  0.667074  0.640243
1851     Kiribati  Male  15+ years  2007  0.653724  0.665278  0.642580
1836     Kiribati  Male  15+ years  2002  0.652328  0.666099  0.637620
1854     Kiribati  Male  15+ years  2008  0.651599  0.663414  0.639374
1512  Timor-Leste  Male  15+ years  2014  0.649498  0.664485  0.634123
1857     Kiribati  Male  15+ years  2009  0.649391  0.661836  0.636715
1833     Kiribati  Male  15+ years  2001  0.649336  0.665080  0.633709

      prevalence_percent
1845           65.604680
...
      location     sex  age_group  year       val     upper     lower  \
15811  Eritrea  Female  15+ years  2010  0.006671  0.008813  0.005015
15814  Eritrea  Female  15+ years  2011  0.006722  0.008809  0.005059
15817  Eritrea  Female  15+ years  2012  0.006776  0.008892  0.005089
15820  Eritrea  Female  15+ years  2013  0.006813  0.008929  0.005093
15823  Eritrea  Female  15+ years  2014  0.006849  0.008858  0.005142
15808  Eritrea  Female  15+ years  2009  0.006860  0.009034  0.005176
15826  Eritrea  Female  15+ years  2015  0.006878  0.008925  0.005168
15829  Eritrea  Female  15+ years  2016  0.006905  0.009001  0.005157
15832  Eritrea  Female  15+ years  2017  0.006926  0.009001  0.005182
15835  Eritrea  Female  15+ years  2018  0.006946  0.009021  0.005176

       prevalence_percent
15811            0.667112
...
```

Krok 28. 10 największych wartości przy warunku na inną kolumnę

Wybrano kobiety w roku 2019, a następnie 10 lokalizacji z najwyższą częstością palenia.

```
df[(df["sex"] == "Female") & (df["year"] == 2019)].nlargest(10, "val")
```

Wynik w konsoli:

```
location     sex  age_group  year       val  \
19708                      Greenland  Female  15+ years  2019  0.423077
19978                          Nauru  Female  15+ years  2019  0.403244
4588                           Serbia  Female  15+ years  2019  0.377615
2068   Micronesia (Federated States of)  Female  15+ years  2019  0.363633
1888                           Kiribati  Female  15+ years  2019  0.351282
```

```
4318                        Montenegro  Female  15+ years  2019  0.326599
8638                             Chile  Female  15+ years  2019  0.326527
3958                           Croatia  Female  15+ years  2019  0.326480
3868                          Bulgaria  Female  15+ years  2019  0.324911
7198                            Greece  Female  15+ years  2019  0.321793

          upper     lower  prevalence_percent
19708  0.484252  0.364911           42.307686
...
```

2.6. Grupowanie i tabele przestawne

Krok 29. Grupowanie i średnia wszystkich kolumn – MultiIndex

Grupowanie po dwóch kolumnach (location, sex) tworzy indeks złożony.

```
grupy = df.groupby(["location", "sex"]).mean(numeric_only=True)
```

Wynik w konsoli:

```
year       val     upper     lower  \
location       sex
Afghanistan    Both    2004.5  0.071025  0.077642  0.065151
               Female  2004.5  0.019593  0.025055  0.015107
               Male    2004.5  0.122482  0.134730  0.111607
Albania        Both    2004.5  0.268444  0.280724  0.256935
               Female  2004.5  0.094446  0.110725  0.079490
               Male    2004.5  0.444275  0.461782  0.426420
Algeria        Both    2004.5  0.173712  0.186538  0.161078
               Female  2004.5  0.020610  0.026713  0.015822
               Male    2004.5  0.325093  0.349598  0.300477
American Samoa Both    2004.5  0.331939  0.356027  0.309480

                       prevalence_percent
...
```

Krok 30. Grupowanie z różnymi agregacjami

Dla kolumny val policzono średnią, a dla upper i lower liczbę wartości i medianę.

```
df.groupby(["location", "sex"]).agg(
    {"val": "mean", "upper": ["count", "median"], "lower": ["count", "median"]})
```

Wynik w konsoli:

```
val upper           lower
                       mean count    median count    median
location       sex
Afghanistan    Both    0.071025    30  0.082106    30  0.067644
               Female  0.019593    30  0.026021    30  0.015451
               Male    0.122482    30  0.142300    30  0.116585
Albania        Both    0.268444    30  0.260998    30  0.241136
               Female  0.094446    30  0.100072    30  0.074065
               Male    0.444275    30  0.437811    30  0.405701
Algeria        Both    0.173712    30  0.182824    30  0.159473
               Female  0.020610    30  0.026562    30  0.015829
               Male    0.325093    30  0.344164    30  0.298294
American Samoa Both    0.331939    30  0.354364    30  0.313038
```

Kroki 31-32. Nazwy i sortowanie indeksu złożonego

Atrybut `index.names` zwrócił ['location', 'sex'], a `sort_index()` posortował indeks według poziomu sex.

```
grupy.index.names
grupy.sort_index(level="sex", ascending=False)
```

Wynik w konsoli:

```
FrozenList(['location', 'sex'])


                                        year       val     upper     lower  \
location                          sex
Zimbabwe                          Male  2004.5  0.315484  0.330235  0.300383
Zambia                            Male  2004.5  0.259194  0.273248  0.244985
Yemen                             Male  2004.5  0.312699  0.330496  0.295292
Western Sub-Saharan Africa        Male  2004.5  0.146274  0.151042  0.141734
Western Europe                    Male  2004.5  0.345808  0.349101  0.342526
Viet Nam                          Male  2004.5  0.528847  0.541446  0.516309
Venezuela (Bolivarian Republic of) Male  2004.5  0.270519  0.294394  0.248714
Vanuatu                           Male  2004.5  0.426253  0.444546  0.407834
Uzbekistan                        Male  2004.5  0.232490  0.248041  0.217693
Uruguay                           Male  2004.5  0.360943  0.379153  0.342920


                                      prevalence_percent
...
```

Kroki 33-34. Tabela przestawna, jej indeksy i kolumny

Utworzono tabelę: wiersze – rok, kolumny – płeć, wartości – średnia częstość palenia. Indeksem są lata 1990-2019, kolumnami Both, Female, Male.

```
pivot = pd.pivot_table(df, values="val", index="year", columns="sex", aggfunc="mean")
pivot.index
pivot.columns
```

Wynik w konsoli:

```
sex       Both    Female      Male
year
1990  0.243041  0.129851  0.358300
1991  0.241406  0.129171  0.355679
1992  0.239777  0.128537  0.353035
1993  0.238219  0.127970  0.350485
1994  0.236700  0.127469  0.347986
1995  0.235224  0.127022  0.345541
1996  0.233833  0.126612  0.343168
1997  0.232497  0.126212  0.340861
1998  0.231176  0.125816  0.338592
1999  0.229767  0.125357  0.336260


Index([1990, 1991, 1992, 1993, 1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001,
       2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013,
       2014, 2015, 2016, 2017, 2018, 2019],
      dtype='int64', name='year')
Index(['Both', 'Female', 'Male'], dtype='str', name='sex')
```

Krok 35. Tabela przestawna z indeksem złożonym

Indeks utworzono z dwóch kolumn: location i year.

```
pd.pivot_table(df, values="val", index=["location", "year"], columns="sex",
aggfunc="mean")
```

Wynik w konsoli:

```
['location', 'year']
sex                  Both    Female      Male
location    year
```

```
Afghanistan 1990  0.031186  0.009786  0.056109
            1991  0.032945  0.010173  0.058860
            1992  0.035322  0.010653  0.062400
            1993  0.037715  0.011172  0.066201
            1994  0.040002  0.011699  0.070084
            1995  0.042519  0.012290  0.074396
            1996  0.045279  0.012946  0.079149
            1997  0.048168  0.013660  0.084137
            1998  0.050759  0.014325  0.088757
            1999  0.053655  0.015081  0.094013
```

2.7. Wykresy

Kroki 36-37. Import pyplot i wykresy w zeszycie

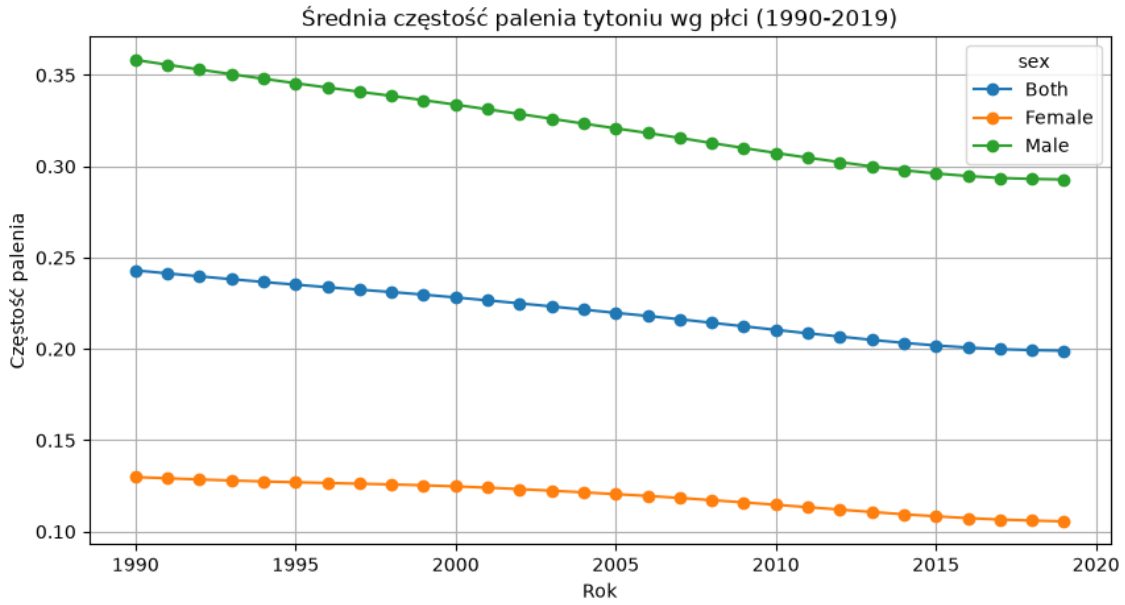
Zaimportowano moduł pyplot, a polecenie `%matplotlib inline` powoduje rysowanie wykresów bezpośrednio pod komórką.

```
import matplotlib.pyplot as plt
%matplotlib inline
```

Krok 38. Wykres na podstawie tabeli przestawnej

Metoda `plot()` tabeli przestawnej rysuje osobną linię dla każdej płci (rys. 1).

```
pivot.plot(figsize=(10, 5), marker="o")
```

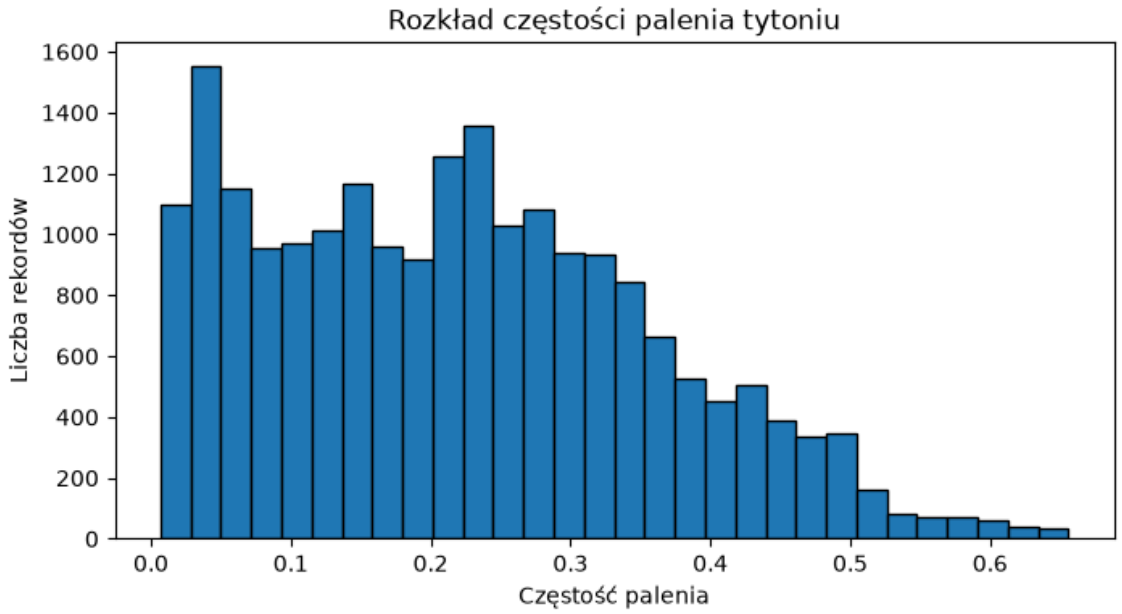


Rys. 1. Średnia częstość palenia tytoniu według płci w latach 1990-2019

Krok 39. Histogram

Histogram pokazuje, ile rekordów ma daną częstość palenia (rys. 2).

```
df["val"].plot(kind="hist", bins=30, edgecolor="black")
```



Rys. 2. Rozkład częstości palenia tytoniu

2.8. Łączenie ramek i dodawanie kolumn

Krok 40. Łączenie ramek – merge i concat

Metodą `merge()` dołączono do każdego wiersza średnią częstość dla danej lokalizacji, a `concat()` skleił wiersze z 1990 i 2019 roku w jedną ramkę.

```
srednie = df.groupby("location", as_index=False)["val"].mean()
polaczone = pd.merge(df, srednie, on="location", how="left")
pd.concat([rok_1990, rok_2019], ignore_index=True)
```

Wynik w konsoli:

```
 location     sex  age_group  year       val     upper     lower  \
0   Global    Male  15+ years  1990  0.450607  0.453930  0.446915
1   Global  Female  15+ years  1990  0.106411  0.108623  0.104415
2   Global    Both  15+ years  1990  0.278179  0.280235  0.276236
3   Global    Male  15+ years  1991  0.447212  0.450214  0.443670
4   Global  Female  15+ years  1991  0.105164  0.107276  0.103293

   prevalence_percent  srednia_kraju
0           45.060696        0.23577
1           10.641115        0.23577
2           27.817878        0.23577
3           44.721154        0.23577
4           10.516446        0.23577
  location     sex  age_group  year       val     upper     lower  \
0   Global    Male  15+ years  1990  0.450607  0.453930  0.446915
1   Global  Female  15+ years  1990  0.106411  0.108623  0.104415
2   Global    Both  15+ years  1990  0.278179  0.280235  0.276236
3   Global    Male  15+ years  2019  0.326673  0.330424  0.322837
4   Global  Female  15+ years  2019  0.066243  0.068325  0.064317
5   Global    Both  15+ years  2019  0.195916  0.198097  0.193681

   prevalence_percent
0           45.060696
1           10.641115
2           27.817878
3           32.667275
4            6.624256
...
```

Krok 41. Nowe kolumny przez operacje matematyczne

Policzono różnicę i stosunek wartości do średniej danej lokalizacji.

```
polaczone["roznica"] = polaczone["val"] - polaczone["srednia_kraju"]
polaczone["stosunek"] = polaczone["val"] / polaczone["srednia_kraju"]
```

Wynik w konsoli:

```
location  year       val  srednia_kraju   roznica  stosunek
0   Global  1990  0.450607        0.23577  0.214837  1.911213
1   Global  1990  0.106411        0.23577 -0.129359  0.451334
2   Global  1990  0.278179        0.23577  0.042409  1.179873
3   Global  1991  0.447212        0.23577  0.211441  1.896811
4   Global  1991  0.105164        0.23577 -0.130606  0.446047
```

Krok 42. Nowe kolumny przez funkcję lambda

Podzielono rekordy na poziom palenia: niski (< 15%), średni (15-30%), wysoki (> 30%).

```
df["poziom"] = df["val"].apply(
    lambda x: "niski" if x < 0.15 else ("średni" if x < 0.30 else "wysoki"))
```

Wynik w konsoli:

```
location  year       val  poziom
0   Global  1990  0.450607  wysoki
1   Global  1990  0.106411   niski
2   Global  1990  0.278179  średni
3   Global  1991  0.447212  wysoki
4   Global  1991  0.105164   niski
poziom
średni    7544
niski     7506
wysoki    5920
Name: count, dtype: int64
```

Wynik: średni – 7 544, niski – 7 506, wysoki – 5 920 rekordów.

2.9. Praca z dużymi plikami

Krok 43. Argument chunksize

Plik wczytano w kawałkach po 5000 wierszy i policzono średnią bez ładowania całego pliku do pamięci.

```
for kawalek in pd.read_csv("wynik_smoking.csv", chunksize=5000):
    suma += kawalek["val"].sum()
    liczba += len(kawalek)
```

Wynik w konsoli:

```
Liczba wierszy: 20970
Średnia val: 0.2205751410672866
```

Wynik: 5 kawałków, 20 970 wierszy, średnia 0.2206 – taka sama jak dla całej ramki.

3. Wnioski

Podczas ćwiczenia poznano podstawy pracy z biblioteką pandas w notatniku Jupyter:

• Tworzenie ramek danych na różne sposoby: ze słownika, z listy list oraz przez wczytanie pliku CSV funkcją read_csv(), a także transponowanie ramki.

• Szybki przegląd danych metodami head(), tail(), info(), shape oraz describe(), która podaje statystyki osobno dla kolumn liczbowych i kategoryzowanych.

• Wykrywanie i usuwanie brakujących wartości metodami isna() i dropna().

• Wybór wierszy i kolumn po nazwach (loc) i po indeksach (iloc), filtrowanie za pomocą warunków logicznych, łączenie warunków operatorem & oraz metodą query(), a także wyszukiwanie słów metodą str.contains().

• Modyfikowanie ramki: dodawanie kolumn z operacji matematycznych i funkcji lambda, usuwanie kolumn (drop) i zmianę ich nazw (rename) oraz zapis wyników do pliku CSV.

• Liczenie statystyk (mean, max, min, unique, value_counts), sortowanie (sort_values) i wybór największych i najmniejszych wartości (nlargest, nsmallest).

• Grupowanie danych (groupby) z różnymi funkcjami agregującymi, pracę z indeksem złożonym (MultiIndex) oraz tworzenie tabel przestawnych (pivot_table).

• Rysowanie wykresów liniowych i histogramów biblioteką matplotlib bezpośrednio w notatniku.

• Łączenie ramek danych metodami merge() i concat().

• Przetwarzanie dużych plików w częściach za pomocą argumentu chunksize.